PBQ-02 Lecture FAQ

QuantInsti Quantitative Learning Pvt Ltd

1. How to fetch historical data for multiple tickers from `yfinance`?

The following script shows an example of how we can do it:

```
import yfinance as yf

script = ['EURINR=X', 'GBPINR=X', 'AEDINR=X', 'INRJPY=X']

df = yf.download(script, period='6mo', interval='1d')

df.head()
```

2. What is the difference between `df.where` an `np.where`?

They both perform similar operations. As we know that pandas is built on top of the numpy library, pandas inherits most of the numpy functionalities. There are some syntactical differences, though.

```
import yfinance as yf

df = yf.download('AAPL', period='6mo')

# np.where example
df[np.where(df['Volume'] > 80000000, True, False)]

# df.where example
df.where(df['Volume'] > 80000000).dropna()
```

3. A lot of the price distributions are not normal; how would we know which one to use?

Generally, we don't use any distribution specifically for prices. However, stock prices usually follow the log-normal distribution, and their returns follows the normal distribution.

4. What are the use cases of tuples in algo trading?

Consider that you are developing an execution algorithm that gets initialized with some parameters. In that case, you can use either list or tuple. If you use a list, there are chances that elements in the list to accidentally change in the rest of the code. To avoid such situations, it is advisable to use tuples which by definition are immutable.

5. Can we create a nested `np.where()`, just as nested if?

Yes, we can create nested conditions using `np.where()`. See the following example:

```
import yfinance as yf

df = yf.download('SBIN.NS', period='1y', interval='1d')

df['ma'] = df['Adj Close'].rolling(20).mean()

# Nested where conditions
df['signals'] = np.where(df['Adj Close'] > df['ma'], 1,
                         np.where(df['Adj Close'] < df['ma'], -1, 0))

df['signals'].value_counts()
```

6. *Can I write `y[y>3]` instead of using `np.where()`?*

You can. But generally, we use `[]` to slice the data. We prefer to use `np.where()` to slice, replace or filter the data structure.

7. *Is there a way to identify all variables and functions defined in a notebook?*

We can use a `%whos` magic command that displays all variables defined inside the current notebook. Please note that only the Jupyter environment provides this functionality. Magic commands do not work outside the Jupyter environment.

8. *Does pandas inherit all functions from numpy?*

As pandas is built on top of numpy, it inherits most of the numpy functionalities. However, there can be syntactical differences in terms of how they are implemented.

9. *What is the difference between `*args` and `**kwargs`?*

We use `*args` (arguments) and `**kwargs` (keyword-arguments) to provide variable length inputs to the functions. We use them when we are unsure of the inputs to functions when we define them. Please refer to section 9.2.4 of the Python handbook for more information on how to use them.